

# API Integration

Connect React Apps to Backend Services

## ■ LEARNING OUTCOME

By the end of this module you will fetch data from REST APIs, use React Query for server state, implement optimistic updates, and handle JWT authentication.

### ■ Skills You Will Gain

- Fetch data and handle loading/error states
- Use React Query for server state management
- Implement mutations and cache invalidation
- Handle JWT authentication
- Use axios with interceptors
- Manage environment variables

■ API integration bridges your React UI with real data. React Query increases developer productivity 2-3x by handling caching, loading states, background refetching, and error handling automatically.

## SECTION 1

# Fetching Patterns and Axios

```
import axios from 'axios'; // Axios instance with interceptors const api = axios.create({ baseURL:
import.meta.env.VITE_API_URL || 'http://localhost:3000/api', timeout: 10000, }); // Add auth token
to every request api.interceptors.request.use(config => { const token =
localStorage.getItem('token'); if (token) config.headers.Authorization = `Bearer ${token}`; return
config; }); // Handle 401 globally api.interceptors.response.use( response => response, error => {
if (error.response?.status === 401) { localStorage.removeItem('token'); window.location.href =
'/login'; } return Promise.reject(error); } ); // Custom useFetch hook function useFetch(url) {
const [state, setState] = useState({ data:null, loading:true, error:null }); useEffect(() => { let
cancelled = false; fetch(url) .then(r => { if(!r.ok) throw new Error(r.statusText); return
r.json(); }) .then(data => { if(!cancelled) setState({data, loading:false, error:null}); })
.catch(e => { if(!cancelled) setState({data:null, loading:false, error:e.message}); }); return ()
=> { cancelled = true; }; }, [url]); return state; }
```

## SECTION 2

# React Query and Auth

```
npm install @tanstack/react-query // Setup import { QueryClient, QueryClientProvider, useQuery,
useMutation, useQueryClient } from '@tanstack/react-query'; const qc = new QueryClient({
defaultOptions: { queries: { staleTime:5*60*1000 } } }); <QueryClientProvider client={qc}><App
/></QueryClientProvider> // useQuery -- fetch and cache function ProductList({ category }) { const
{ data, isLoading, isError, error } = useQuery({ queryKey: ['products', category], queryFn: () =>
api.get(`/products?cat=${category}`).then(r=>r.data), enabled: !!category, }); if (isLoading)
return <Skeleton />; if (isError) return <Error>{error.message}</Error>; return <div>{data.map(p =>
<ProductCard key={p.id} product={p}/>)}</div>; } // useMutation -- create/update/delete function
AddProduct() { const qc = useQueryClient(); const mut = useMutation({ mutationFn: data =>
api.post('/products', data).then(r=>r.data), onSuccess: () => qc.invalidateQueries({
queryKey:['products'] }), }); return <button onClick={() => mut.mutate({name:'New',price:999})}
disabled={mut.isPending}> {mut.isPending ? 'Adding...' : 'Add Product'} </button>; } // JWT Auth
Context const AuthCtx = createContext(null); function AuthProvider({ children }) { const [user,
setUser] = useState(null); const [token, setToken] = useState(() => localStorage.getItem('token'));
useEffect(() => { if (token) { try { const payload = JSON.parse(atob(token.split('.')[1])); if
(payload.exp*1000 < Date.now()) logout(); else setUser(payload); } catch { logout(); } } },
[token]); const login = async (creds) => { const { data } = await api.post('/auth/login', creds);
localStorage.setItem('token', data.token); setToken(data.token); }; const logout = () => {
localStorage.removeItem('token'); setToken(null); setUser(null); }; return <AuthCtx.Provider
value={{ user, login, logout, isAuth:!!token }}>{children}</AuthCtx.Provider>; } const useAuth = ()
=> useContext(AuthCtx); // .env: VITE_API_URL=https://api.example.com (VITE_ prefix required!)
```

## ■ PRACTICE Q & A

**Q1.** What is React Query and why is it better than useEffect for data fetching?

**A:** React Query provides: auto caching, background refetch, loading/error states, deduplication, optimistic updates, cache invalidation. useEffect requires implementing all of this manually.

**Q2.** What is a queryKey in React Query?

**A:** Array that uniquely identifies a query and its cache entry. ['products'] and ['products', categoryId] are different entries. invalidateQueries(['products']) refetches all matching queries.

**Q3.** What is staleTime in React Query?

**A:** How long data is considered fresh -- no background refetch triggered. After staleTime, next mount of the component triggers a background refetch while showing cached data.

**Q4.** Where should you store JWT tokens?

*A: localStorage: XSS risk but CSRF-safe. httpOnly cookies: XSS-safe but needs CSRF protection. For most SPAs, localStorage with short expiry is acceptable. Never in React state (lost on refresh).*

**Q5.** What does queryClient.invalidateQueries do?

*A: Marks matching queries as stale and triggers a background refetch. Call after mutations to keep UI in sync with server. Pass the query key prefix to match related queries.*

| ■ API Integration Cheat Sheet                 |                                 |
|-----------------------------------------------|---------------------------------|
| <code>axios.create({baseUrl, timeout})</code> | Create axios instance           |
| <code>api.interceptors.request.use(fn)</code> | Add auth header to requests     |
| <code>useQuery({queryKey, queryFn})</code>    | Fetch and cache data            |
| <code>queryKey: ['products', filter]</code>   | Include all params in key       |
| <code>enabled: !!id</code>                    | Only fetch when condition true  |
| <code>useMutation({mutationFn})</code>        | Create/update/delete            |
| <code>qc.invalidateQueries(key)</code>        | Refetch after mutation          |
| <code>mut.isPending</code>                    | Mutation in-progress state      |
| <code>import.meta.env.VITE_API_URL</code>     | Environment variable            |
| <code>VITE_ prefix only</code>                | Client-side env vars            |
| <code>staleTime: 5*60*1000</code>             | 5 min before background refetch |
| <code>isLoading / isError / data</code>       | Query state properties          |